

교과목 2 : 데이터 분석과 머신러닝 / 딥러닝

단원 2 : 머신러닝

DAY5. 머신러닝 알고리즘 및 앙상블

목 차

1. 머신러닝 알고리즘 분류		3
2. 앙상블의 이해		5
3. 앙상블 모델 적용 코드 분석		9
4. 신경망(ANN)		17

1. 머신러닝 알고리즘 분류

머신러닝 알고리즘을 크게 분류하면 다음과 같습니다.

1. 지도학습 (Supervised Learning)

정답(Label)이 있는 데이터를 학습

- 선형회귀(Linear Regression)
- 로지스틱 회귀(Logistic Regression)
- 의사결정트리(Decision Tree)
- SVM(Support Vector Machine)
- KNN
- 신경망(Neural Network)

앙상블 알고리즘

여러 개의 모델을 결합하여 성능을 높이는 방법

- Random Forest
- AdaBoost
- Gradient Boosting
- XGBoost
- LightGBM
- CatBoost

2. 비지도학습 (Unsupervised Learning)

정답(Label)이 없는 데이터를 학습

클러스터링(군집화)

비슷한 데이터끼리 그룹으로 묶음

- K-Means
- Hierarchical Clustering
- DBSCAN
- Mean Shift
- Gaussian Mixture Model

3. 차원 축소 (Dimensionality Reduction)

- PCA
- LDA
- t-SNE
- UMAP

2. 앙상블의 이해

앙상블(Ensemble Learning)의 이해

머신러닝 모델을 개발하다 보면 하나의 알고리즘만으로는 원하는 수준의 예측 성능을 얻기 어려운 경우가 많다. 특히 의사결정트리(Decision Tree)는 구조가 단순하고 이해하기 쉽다는 장점이 있지만, 데이터에 지나치게 맞춰 학습하는 과적합(Overfitting) 문제가 발생할 수 있다. 또한 학습 데이터가 조금만 바뀌어도 생성되는 트리 구조가 크게 달라질 수 있어 안정성이 떨어지는 단점이 있다. 이러한 문제를 해결하기 위해 등장한 기법이 바로 **앙상블 학습(Ensemble Learning)**이다. 제공된 강의자료에서도 의사결정트리의 단점을 보완하기 위한 방법으로 Random Forest, Bagging, Boosting을 소개하고 있다.

앙상블은 여러 개의 머신러닝 모델을 결합하여 하나의 강력한 모델을 만드는 방법이다. 사람의 의사결정 과정에 비유하면 한 명의 전문가 의견만 듣는 것이 아니라 여러 전문가의 의견을 종합하여 최종 결정을 내리는 것과 같다. 개별 모델은 일부 실수를 할 수 있지만, 여러 모델의 결과를 종합하면 오류가 줄어들고 더 안정적인 결과를 얻을 수 있다.

의사결정트리와 앙상블의 관계

신용 대출 신청자의 채무 불이행 여부를 예측하기 위해 의사결정트리를 활용하였다. 예를 들어 다음과 같은 정보가 있다고 가정해 보자.

- 신청자의 나이
- 현재 계좌 잔고
- 과거 대출 이력
- 대출 금액
- 신용 등급

의사결정트리는 이러한 정보를 이용하여 "채무를 정상적으로 상환할 가능성이 높은가?" 또는 "채무 불이행 가능성이 높은가?"를 판단한다.

하지만 하나의 의사결정트리만 사용하면 특정 데이터에 지나치게 의존하게 되어 새로운 데이터에 대한 예측 성능이 낮아질 수 있다. 따라서 여러 개의 트리를 만들어 결과를 종합하는 앙상블 기법이 활용된다.

Bagging(Bootstrap Aggregating)

Bagging은 가장 기본적인 앙상블 기법이다. 원본 데이터에서 중복을 허용하면서 여러 개의 샘플 데이터를 생성하고, 각 데이터로 별도의 의사결정트리를 학습시킨다.

예를 들어 1,000건의 대출 데이터가 있다고 가정하면, 이 데이터를 여러 번 무작위 추출하여 새로운 학습 데이터를 생성한다. 생성된 각각의 데이터셋으로 독립적인 의사결정트리를 학습시키고, 최종 예측 시에는 모든 트리의 결과를 종합한다.

분류 문제에서는 일반적으로 다수결(Voting)을 사용한다. 예를 들어 다섯 개의 트리 중 세 개가 "대출 승인"을 예측하고 두 개가 "대출 거절"을 예측한다면 최종 결과는 "대출 승인"이 된다.

Bagging의 가장 큰 장점은 모델의 분산(Variance)을 줄여 안정적인 예측 결과를 제공한다는 점이다. 또한 과적합을 감소시키는 효과가 있다.

Random Forest

Random Forest는 Bagging을 발전시킨 대표적인 앙상블 알고리즘이다.

Bagging에서는 모든 특성을 사용하여 트리를 생성하지만, Random Forest는 트리를 생성할 때 사용할 특성(Feature)을 무작위로 선택한다.

예를 들어 대출 심사 데이터에 다음과 같은 특성이 있다고 가정하자.

- 나이
- 소득
- 계좌 잔고
- 직업
- 학력
- 신용등급

Random Forest는 각 트리를 생성할 때 이러한 특성 중 일부만 선택하여 학습한다.

첫 번째 트리는

- 나이
- 소득
- 신용등급

만 사용할 수 있고,

두 번째 트리는

- 직업

- 학력
- 계좌 잔고

만 사용할 수 있다.

이처럼 각 트리가 서로 다른 관점에서 학습하므로 모델 간 다양성이 증가하고, 결과적으로 예측 성능이 향상된다.

Random Forest는 현재 금융권 신용평가, 의료 진단, 제조업 품질관리, 고객 이탈 예측 등 다양한 분야에서 널리 사용되는 대표적인 앙상블 기법이다.

Boosting

Boosting은 Bagging과 접근 방식이 다르다.

Bagging은 여러 모델을 독립적으로 학습시키지만, Boosting은 이전 모델이 틀린 데이터를 다음 모델이 집중적으로 학습하도록 한다.

즉,

첫 번째 트리가 학습을 수행한 후 오답을 분석하고,

두 번째 트리는 첫 번째 트리가 틀린 데이터를 더 중요하게 학습한다.

세 번째 트리는 앞선 모델들이 틀린 데이터를 다시 학습한다.

이 과정을 반복하면서 모델의 성능을 점진적으로 향상시킨다.

따라서 Boosting은 어려운 문제에 대한 학습 능력이 매우 뛰어나며, 최근 머신러닝 경진대회에서도 높은 성능을 보여주는 대표적인 방법이다.

AdaBoost

AdaBoost(Adaptive Boosting)는 가장 초기의 Boosting 알고리즘이다.

강의자료에서도 의사결정트리의 성능을 향상시키기 위한 방법으로 적응형 부스팅(Adaptive Boosting)이 소개되어 있다.

AdaBoost는 이전 모델이 틀린 데이터에 더 높은 가중치를 부여하고 다음 모델이 해당 데이터를 집중적으로 학습하도록 만든다.

학습이 진행될수록 어려운 데이터에 대한 분류 능력이 향상되며, 최종적으로 여러 모델의 결과를 가중합하여 최종 예측을 수행한다.

Gradient Boosting

Gradient Boosting은 AdaBoost를 더욱 발전시킨 알고리즘이다.

AdaBoost가 잘못 분류된 데이터에 집중하는 방식이라면, Gradient Boosting은 실제값과 예측값

의 차이인 오차(Error)를 학습한다.

즉,

오차 = 실제값 - 예측값

을 계산한 뒤, 다음 모델이 이 오차를 줄이도록 학습한다.

이를 반복하면서 예측 성능을 점진적으로 향상시킨다.

대표적인 알고리즘으로는

- XGBoost
- LightGBM
- CatBoost

가 있다.

현재 산업 현장에서 가장 많이 사용되는 앙상블 기법 중 하나이다.

신용위험 예측 사례와 앙상블

독일 신용 데이터(German Credit Data)를 활용하여 채무 불이행 여부를 예측하는 의사결정트리 모델을 구축하였다.

초기 의사결정트리 모델의 테스트 정확도는 약 73% 수준으로 나타났으며, 채무 불이행자를 정확하게 예측하는 비율이 충분히 높지 않았다.

이를 개선하기 위해 적응형 부스팅(Adaptive Boosting)을 적용하였다.

그 결과

- 정확도 : 73% → 75%
- 오류율 : 27% → 25%

로 향상되었다.

이는 여러 개의 의사결정트리를 결합하는 앙상블 기법이 단일 의사결정트리보다 높은 예측 성능을 제공할 수 있음을 보여주는 좋은 사례이다.

앙상블은 여러 개의 머신러닝 모델을 결합하여 단일 모델보다 높은 성능을 얻기 위한 방법이다. Bagging은 여러 개의 모델을 독립적으로 생성하여 결과를 종합하는 방식이며, Random Forest는 Bagging에 랜덤 특성 선택을 추가하여 성능을 향상시킨 기법이다. Boosting은 이전 모델이 틀린 데이터를 다음 모델이 집중적으로 학습하도록 하여 예측 성능을 점진적으로 향상시키는 방법이다. 현재 실제 산업 현장에서는 Random Forest, XGBoost, LightGBM, CatBoost와 같은 앙상블 기법이 금융, 의료, 제조, 마케팅, 추천 시스템 등 다양한 분야에서 핵심 예측 모델로 활용되고 있다.

3. 앙상블 모델 적용 코드 분석

의사결정트리 기반 앙상블 모델 적용 코드 분석

이번 실습에서는 기존의 PyTorch 기반 의사결정트리 모델에 다양한 앙상블(Ensemble) 기법을 추가하여 성능을 비교하였다. 기존 모델은 하나의 의사결정트리만 사용하여 예측을 수행하였지만, 추가된 코드에서는 Bagging, Random Forest, AdaBoost, Gradient Boosting 기법을 각각 적용하여 성능을 향상시키도록 구성하였다.

앙상블의 기본 철학은 단순하다. 하나의 모델이 내린 판단보다 여러 모델의 판단을 종합한 결과가 더 정확할 가능성이 높다는 것이다. 현실에서도 중요한 의사결정을 할 때 여러 전문가의 의견을 종합하듯이 머신러닝에서도 여러 모델의 결과를 결합하여 더 좋은 성능을 얻는다.

1. Bagging(Bootstrap Aggregating) 코드와 수식 설명

Bagging은 앙상블 기법 중 가장 이해하기 쉬운 방법이다.

추가된 코드에서는 다음과 같은 구조로 구현되어 있다.

```
from sklearn.ensemble import BaggingClassifier
```

```
bag_model = BaggingClassifier(  
    estimator=DecisionTreeClassifier(),  
    n_estimators=100,  
    random_state=42  
)
```

여기서 `n_estimators=100`은 의사결정트리를 100개 생성하라는 의미이다.

Bagging의 핵심은 Bootstrap Sampling이다.

원본 데이터가 다음과 같다고 가정하자.

$$D = \{x_1, x_2, x_3, \dots, x_n\}$$

Bagging은 원본 데이터에서 중복을 허용하여 여러 개의 새로운 학습 데이터를 만든다.

$$D_1, D_2, D_3, \dots, D_{100}$$

예를 들어 1000개의 데이터가 있다면 첫 번째 트리는

1, 3, 5, 7, 7, 10 ...

과 같은 데이터를 사용하고,

두 번째 트리는

2, 4, 4, 6, 8 ...

와 같은 데이터를 사용한다.

각 데이터셋마다 별도의 의사결정트리를 생성한다.

Tree1

Tree2

Tree3

...

Tree100

각 트리의 예측 함수를 다음과 같이 표현한다.

$h1(x), h2(x), h3(x), ..., h100(x)$

분류 문제에서는 다수결을 사용한다.

수식은 다음과 같다.

$H(x) = \operatorname{argmax}_c \sum I(h_i(x)=c)$

여기서

- $H(x)$ 는 최종 예측값
- $h_i(x)$ 는 개별 트리의 예측
- $I()$ 는 조건이 참이면 1, 거짓이면 0

을 의미한다.

예를 들어

Tree1 → YES

Tree2 → YES

Tree3 → NO

Tree4 → YES

Tree5 → NO

이면 YES가 3표, NO가 2표이므로 최종 결과는 YES가 된다.

Bagging의 장점은 의사결정트리의 분산(Variance)을 감소시키는 것이다. 단일 트리는 데이터 변화에 매우 민감하지만, 여러 트리의 결과를 평균화하면 특정 트리의 실수가 전체 결과에 미치는 영향이 줄어든다.

2. Random Forest 코드와 수식 설명

추가된 Random Forest 코드는 다음과 같은 형태이다.

```
from sklearn.ensemble import RandomForestClassifier
```

```
rf_model = RandomForestClassifier(  
    n_estimators=100,  
    max_depth=10,  
    random_state=42  
)
```

Random Forest는 Bagging을 발전시킨 모델이다.

Bagging에서는 모든 특성을 사용하여 트리를 생성하지만, Random Forest는 트리를 생성할 때 사용할 특성도 무작위로 선택한다.

예를 들어 데이터가 다음 특성으로 구성되어 있다고 하자.

나이

소득

신용등급

대출금액

계좌잔고

직업

일반 의사결정트리는 모든 특성을 사용한다.

반면 Random Forest는

Tree1

나이

신용등급

직업

Tree2

소득

대출금액

계좌잔고

Tree3

나이

소득

직업

처럼 일부 특성만 사용한다.

이렇게 하면 트리들이 서로 다른 관점으로 학습하게 된다.

최종 예측 공식은 다음과 같다.

분류 문제

$$H(x) = \text{mode}\{h_1(x), h_2(x), \dots, h_M(x)\}$$

회귀 문제

$$H(x) = \frac{1}{M} \sum_{m=1}^M h_m(x)$$

여기서

- M은 트리 개수
- $h(x)$ 는 각 트리의 예측값

이다.

코드에서

n_estimators=100

은 트리를 100개 생성한다는 의미이다.

max_depth=10

은 트리 깊이를 최대 10단계로 제한하여 과적합을 방지한다.

Random Forest는 실제 산업 현장에서 가장 많이 사용하는 머신러닝 알고리즘 중 하나이며, 성능과 안정성의 균형이 뛰어나다.

3. AdaBoost 코드와 수식 설명

추가된 AdaBoost 코드는 다음과 같다.

```
from sklearn.ensemble import AdaBoostClassifier
```

```
ada_model = AdaBoostClassifier(  
    n_estimators=100,  
    learning_rate=1.0,  
    random_state=42  
)
```

AdaBoost의 핵심은

틀린 문제를

점점 더 중요하게

학습한다.

이다.

초기에는 모든 데이터의 중요도가 동일하다.

수식은 다음과 같다.

$$w_i = \frac{1}{N}$$

예를 들어

100개 데이터

가 있다면

모든 데이터 중요도 = 0.01

이다.

첫 번째 트리를 학습한 뒤 오류율을 계산한다.

$$\epsilon_t = \sum_{i=1}^N w_i I(y_i \neq h_t(x_i))$$

오류율이 낮을수록 해당 모델은 좋은 모델이다.

AdaBoost는 모델의 중요도를 계산한다.

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$$

예를 들어

오류율 = 0.2

이면

$\alpha \approx 0.693$

이다.

오류율이 더 낮으면 α 는 더 커진다.

즉,

잘 맞추는 모델

=

큰 영향력

못 맞추는 모델

=

작은 영향력

을 갖게 된다.

데이터 가중치 갱신 공식은 다음과 같다.

$$w_i \leftarrow w_i \exp(-\alpha_t y_i h_t(x_i))$$

이 과정에서 틀린 데이터의 가중치는 증가하고 맞힌 데이터의 가중치는 감소한다.

최종 예측은 다음과 같다.

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$$

즉, 여러 모델의 결과를 단순 평균하지 않고 성능이 좋은 모델에 더 높은 가중치를 부여한다.

4. Gradient Boosting 코드와 수식 설명

추가된 Gradient Boosting 코드는 다음과 같다.

```
from sklearn.ensemble import GradientBoostingClassifier
```

```
gb_model = GradientBoostingClassifier(
```

```
    n_estimators=100,
```

```
    learning_rate=0.1,
```

```
    max_depth=3
```

```
)
```

Gradient Boosting은 AdaBoost보다 발전된 방식이다.

AdaBoost는 틀린 데이터에 집중한다.

Gradient Boosting은

오차 자체를 학습

한다.

첫 번째 모델의 예측값을 계산한다.

$\hat{y}_1$

실제값과의 차이를 계산한다.

$$r_i = y_i - \hat{y}_i$$

예를 들어

실제값 = 80

예측값 = 70

이라면

오차 = 10

이다.

두 번째 모델은 원래 정답을 학습하는 것이 아니라 이 오차 10을 학습한다.

Gradient Boosting은 손실함수의 기울기를 이용한다.

손실함수는 다음과 같다.

$$L(y, F(x))$$

기울기는 다음과 같다.

$$g_i = -\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}$$

다음 트리는 이 기울기를 예측한다.

모델 업데이트 공식은 다음과 같다.

$$F_m(x) = F_{m-1}(x) + \eta h_m(x)$$

여기서

- $F_m(x)$ 는 현재 모델
- $F_{m-1}(x)$ 는 이전 모델
- $h_m(x)$ 는 새 트리
- η 는 learning_rate

이다.

실제로 코드의

learning_rate=0.1

은 새 트리가 기존 모델에 10%만 영향을 주도록 설정하는 의미이다.

최종 모델은 다음과 같이 표현된다.

$$F(x) = \sum_{m=1}^M \eta h_m(x)$$

즉, 여러 트리가 조금씩 오차를 수정하면서 최종 모델을 완성한다.

5. 성능 비교 코드 설명

추가된 모든 노트북에는 기존 Decision Tree와 앙상블 모델을 비교하는 평가 코드가 포함되어 있다.

대표적으로 정확도는 다음 공식으로 계산된다.

$$Accuracy = \frac{\text{정확히 예측한 샘플 수}}{\text{전체 샘플 수}}$$

예를 들어

100개 중 85개 정답

이면

Accuracy = 0.85

이다.

또한 Precision, Recall, F1-Score도 출력된다.

정밀도(Precision)

$$Precision = \frac{TP}{TP+FP}$$

재현율(Recall)

$$Recall = \frac{TP}{TP+FN}$$

F1-Score

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

성능 비교 코드의 목적은 단일 의사결정트리와 Bagging, Random Forest, AdaBoost, Gradient Boosting이 얼마나 성능 차이를 보이는지 정량적으로 확인하는 것이다.

일반적으로는 다음과 같은 경향이 나타난다.

Decision Tree

↓

Bagging

↓

Random Forest

↓

AdaBoost

↓

Gradient Boosting

순으로 성능이 향상되는 경우가 많지만, 데이터 특성에 따라 결과는 달라질 수 있다. 따라서 추가된 노트북의 성능 비교 코드는 단순히 정확도를 출력하는 것이 아니라, 어떤 앙상블 기법이 현재 데이터에 가장 적합한지를 확인하는 실험 과정이라고 이해하는 것이 중요하다.

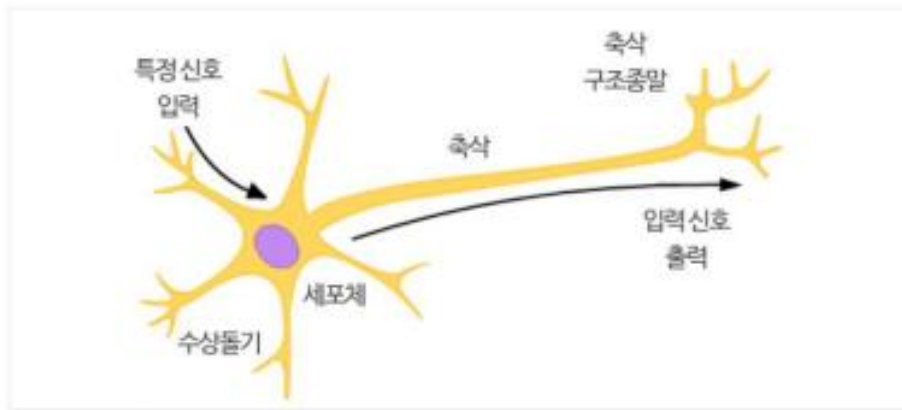
4. 신경망(ANN)

생물학적 뉴런을 인공 뉴런으로 확장하기

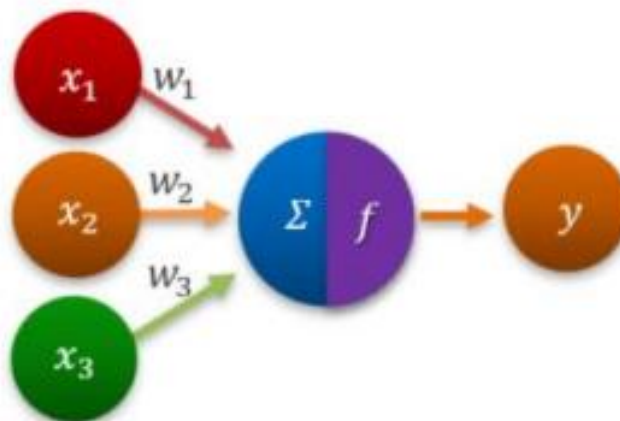
1. 신경망 구조의 이해

1) 인공 신경망 구조

- 인공 신경망(Artificial Neural Network) : 일련의 입력 신호와 출력 신호 관계 모델링
- 생물학적 뇌 : 감각 입력의 자극에 대한 반응을 이해한 후 인공 신경망으로 유도한 모델
- 뉴런(neurons)이라는 상호 연결된 세포 네트워크 사용
→ 뇌의 대규모 학습 용량 제공
- 이해하기 어려운 학습 문제 → ANN : 인공 뉴런의 네트워크 활용
- 생물학적 뉴런의 기술 묘사
 - * 축삭 : 신경 세포를 중심으로 길게 뻗어 나온 돌기



- 인공 뉴런의 생물학적 구조와 기능 설계



생물학적 뉴런을 인공 뉴런으로 확장하기

2) ANN(Artificial Neural Network) 활용

- 포털 사이트의 검색 엔진 등
- 스마트폰 앱의 음성 서비스
- 필기, 이미지 인식 프로그램
- 우편물 정렬 기계
- 건물 환경 등의 제어
- 스마트 장치의 자동화
- 날씨와 기후 패턴 학습
- 인장 강도 및 유체 역학
- 사회, 경제, 과학 현상의 모델
- 수치 예측, 분류, 자율 패턴 인식 등

3) ANN(Artificial Neural Network) 출력

- 입력 데이터와 출력 데이터의 정의는 있지만 단순함
- 입력과 출력이 연관된 프로세스를 복잡한 문제 유형으로 적용
- 블랙박스 방법으로 작동함

생물학적 뉴런을 인공 뉴런으로 확장하기

4) 생물학적 뉴런

- 인간 두뇌 활동의 개념 모델로 설계됨
- 생물학적 뉴런의 기능에 대한 이해
- 입력 신호 : 생화학적 과정에 따라 세포의 수상돌기를 통해 전달
- 자극의 상대적인 중요도 또는 빈도에 따라 중요 가중치 부여
- 세포체에서 입력된 신호를 축적해 세포가 발화한 임계값에 도달했을 때 출력 신호를 보냄
- 전기 화학적 과정에 따라 축삭으로 전달
- 축삭이 종료된 후 다시 전기적 신호를 화학적 신호로 처리
- 시냅스라고 하는 좁은 간격을 통과해서 근처 이웃 뉴런으로 전달

5) 인공 뉴런 모델

- 생물학적 모델과 유사한 용어로 이해
- 방향성 네트워크 다이어그램
 - * 수상 돌기의 입력 신호(x 변수)와 출력 신호(y 변수) 사이의 관계
- 각각의 수상돌기 신호를 중요도에 따라 가중치(weight)부여
- 입력 신호가 세포체에서 추가되고 다시 f 로 표현하는 활성화 함수(Activation Function) 결과에 전달

생물학적 뉴런을 인공 뉴런으로 확장하기

6) 인공 뉴런 신경망 변형

- 신경망 변형
- 활성화 함수
 - * 뉴런의 순방향 입력 신호를 하나의 출력 신호로 반환
 - * 네트워크의 보다 먼 거리까지 확산
- 네트워크 토폴로지(topology)
 - * 모델의 뉴런 수와 계층 수
 - * 연결 방식을 설명함
- 훈련 알고리즘
 - * 입력 신호의 비례에 따라 뉴런 억제
 - * 연결 가중치를 설정하는 방식을 활용하여 훈련 알고리즘 활성화

생물학적 뉴런을 인공 뉴런으로 확장하기

2. 신경망의 방식

1) 인공 뉴런 신경망 구조

◦ 활성화 함수

- * 인공 뉴런으로 입력되는 정보 처리
- * 네트워크를 통해 정보를 전달해가는 매커니즘
- * 인공 뉴런 : 생물학적 버전 모델링
- * 자연의 설계에 적합하도록 모델링
 - 전체 입력 신호를 추가하고 발화 임계값으로 만족 여부 판단
 - 임계값 충족 : 뉴런의 신호로 전달
 - 임계값 불충족 : 아무것도 하지 않음
- * 인공 뉴런 : 생물학적 버전 모델링
- * ANN(Artificial Neural Network) 임계값 활성화 함수
 - 지정된 입력이 임계값에 도달할 경우 출력 신호 생성

2) 인공 뉴런 신경망 네트워크

◦ 네트워크 토폴로지(topology)

- * 신경망의 학습 능력은 상호 연결된 상태의 뉴런 토폴로지에 따라 달라짐
- * 주요 특성
 - 계층 개수
 - 네트워크 정보의 역방향 이동 여부
 - 네트워크의 각 계층별 노드 개수
 - 토폴로지 : 네트워크로 학습 가능한 작업의 복잡도
 - 네트워크 역량 : 네트워크의 크기 함수, 네트워크의 구성 단위로 배열되는 방식

생물학적 뉴런을 인공 뉴런으로 확장하기

3) 인공 뉴런 신경망 변형

◦ 계층 개수

- * 네트워크 토폴로지로 구분됨
- * 입력 노드 : 뉴런의 입력 데이터에서 미처리 신호를 받음
- * 각 입력 노드 : 데이터 셋에서 한 개의 특징을 전달 처리함
- * 입력 노드가 보낸 신호는 출력 노드가 받음(입력 데이터를 수신한 상태로 처리)
- * 출력 노드는 자체 활성화 함수로 최종 예측
- * 입력과 출력 노드는 계층으로 구분된 그룹에 배열됨
- * 단층 네트워크 : 네트워크 한 개에 연결된 가중치 집합만 포함
- * 다중 네트워크
 - 한 개 이상의 은닉 계층을 추가한 후 출력 노드에 도착하기 전에 입력 노드에서 수신한 신호 처리
 - 한 계층의 전체 노드가 다음 계층의 노드와 완전하게 연결됨

생물학적 뉴런을 인공 뉴런으로 확장하기

4) 인공 뉴런 신경망 정보 이동

◦ 정보 이동 방향

* 피드포워드(feed forward) 네트워크

- 정보 이동이 한 방향으로 연결되어 입력 신호가 출력 노드에 도달함
- 유연성 제공
- 계층 수와 계층별 노드 수가 변함

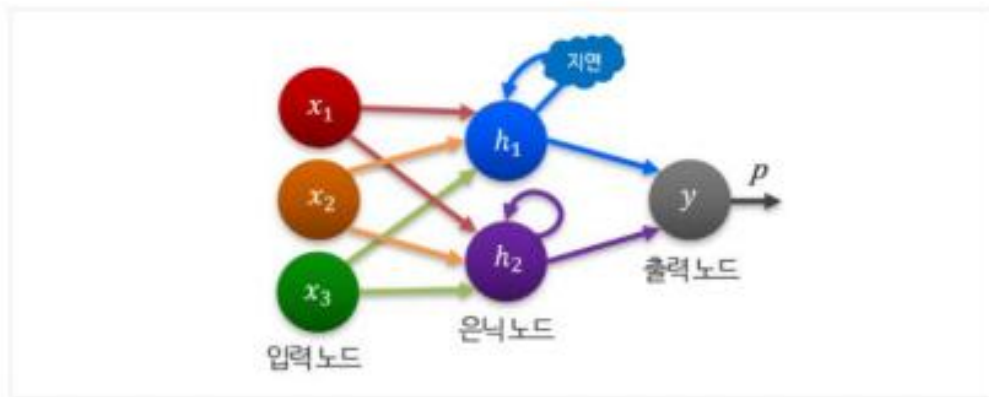
* 다중 출력 : 모델링 또는 다중 은닉 계층 적용

* 심층 신경망(Deep Network) : 다중 은닉 계층을 포함하는 신경망

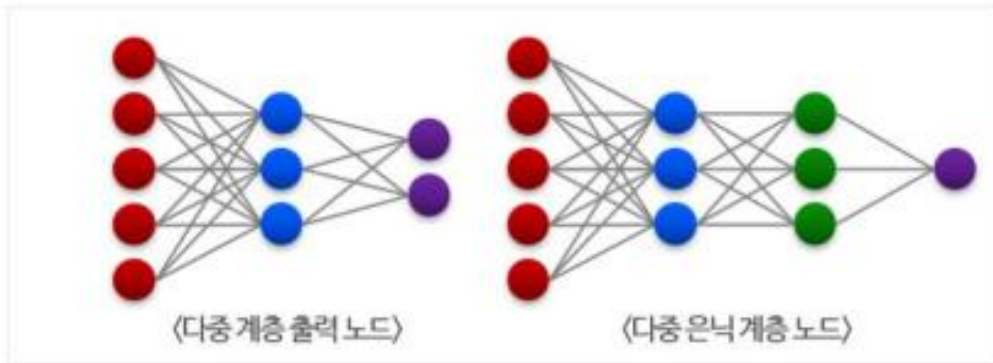
* 신경망의 훈련은 딥러닝에 해당

* 데이터 셋의 훈련된 심층 신경망은 이미지 인식 또는 텍스트 처리 시 사람과 유사한 능력

* 정보의 역방향 흐름 네트워크의 시간 지연 모델링



* ANN(Artificial Neural Network) 다중 출력 노드 또는 다중 은닉 계층



생물학적 뉴런을 인공 뉴런으로 확장하기

5) 인공 뉴런 신경망 노드

- 계층별 노드 개수
 - * 계층 개수와 정보 이동 방향의 변형
 - * 신경망 : 각 계층의 노드 개수
 - * 복잡도의 변화에 영향을 줄 수 있음
- 입력 노드 개수
 - * 입력 데이터의 특정 개수로 사전에 결정
 - 출력 노드의 개수=모델링 된 출력 개수
 - * 출력 클래스 레벨 개수로 사전에 미리 결정
 - * 검증 데이터 셋에 대해 적합한 성능을 표시하는 최소한의 노드 사용
 - * 신경망은 적은 개수의 은닉 노드만으로 뛰어난 학습 능력 제공 가능

6) 인공 뉴런 신경망 훈련

- 전파로 신경망을 훈련함
 - * 오류를 반대로 전파시키는 전략
 - * 역전파 알고리즘은 다층 피드 포워드 네트워크로 데이터마이닝 분야에서 보편화 됨

7) 인공 뉴런 신경망 방식의 장점

- 역전파로 신경망을 훈련할 경우의 장점
 - * 분류 또는 수치 예측 문제에 적응
 - * 다른 유형의 알고리즘보다도 더 복잡한 패턴의 모델링
 - * 데이터의 근본적인 관계는 가정하지 않음

생물학적 뉴런을 인공 뉴런으로 확장하기

8) 인공 뉴런 신경망 방식의 단점

- 역전파로 신경망을 훈련할 경우의 단점
 - * 훈련이 매우 계산 집약적이고 느림
 - * 네트워크 토폴로지가 복잡할 경우 느림
 - * 훈련 데이터의 과적합 가능성이 큼
 - * 해석하기 어렵고 복잡한 블랙박스 모델을 생성함

9) 인공 뉴런 신경망 방식

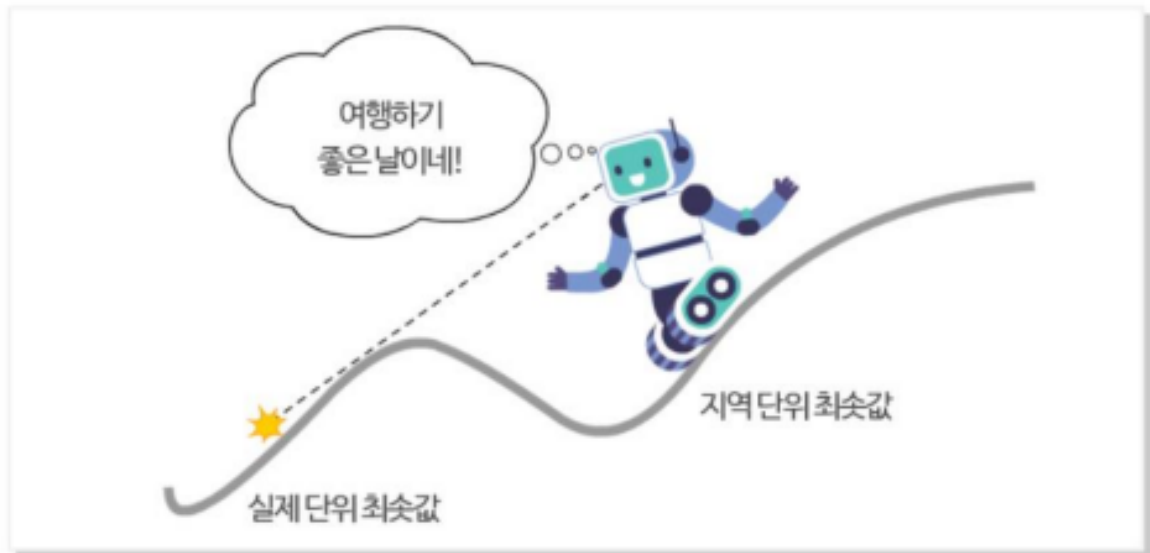
- 역전파 알고리즘의 단계
 - * 순방향 단계
 - 뉴런이 점차적으로 활성화 됨(입력 계층 → 출력 계층)
 - 뉴런의 활성 함수와 가중치가 적용됨
 - 계층에 도달했을 때 출력 신호 생성
 - * 역방향 단계
 - 순방향에서 생성된 네트워크 출력 신호를 훈련 데이터의 목표 데이터 값과 비교
 - 네트워크 출력 신호와 실제값의 차이로 인한 오차 발생
 - 네트워크에서 역방향으로 전파되었을 때 뉴런 사이의 연결 가중치를 수정하여 예측의 오차를 감소시킴

생물학적 뉴런을 인공 뉴런으로 확장하기

9) 인공 뉴런 신경망 방식

◦ 경사 하강법 알고리즘

- * 역전파 알고리즘으로 각 뉴런의 활성화 함수 미분
- * 입력된 각 가중치의 방향으로 경사 식별
- * 경사는 가중치의 변화에 따라 방향 표시(오차의 감소 또는 증가 방향)
- * 알고리즘은 가중치를 학습률 만큼 변경 적용하여 오차를 최대한 감소시킴
- * 학습률이 커질수록 알고리즘은 더 빠른 속도로 지나칠 위험을 감소시킴 (훈련 시간 감소)
- * 경사 하강 알고리즘의 최소 오차를 검색하면서 지역의 최소 위치값 찾기



인공신경망을 활용한 콘크리트 강도의 모델링

1. 콘크리트 강도 모델링의 준비

1) ANN(Artificial Neural Network)의 콘크리트 강도성 모델링 준비

- 최종 산출물의 정확한 강도 예측이 어려움
- 투입 자재의 구성 목록 : 콘크리트의 강도를 명확하게 예측할 수 있는 모델로 더 안전한 건설 사례 구성

2) 1단계 : 분석 데이터 수집

- 콘크리트 내압 강도 데이터의 활용
 - * 데이터 셋 : 1,030개의 콘크리트 예시
 - * 구성 요소를 주로 설명하는 8개의 특징
 - * 특징은 최종 내압 강도와 관련됨
 - 시멘트, 숙성 시간, 슬래그, 재, 물, 굵은 골재, 고성능 감수재, 작은 골재의 양 등

3) 2단계 : 분석 데이터 탐색과 준비

- 정규화 또는 표준화 함수로 데이터 재조정
- 데이터의 75%는 훈련 집합, 25%는 테스트 집합으로 구분
- 훈련 데이터 셋 → 신경망 구축
- 테스트 데이터 셋 → 모델의 미래 결과가 일반화가 잘 될지 평가
- 신경망은 과적합되기 쉬우므로 중요함

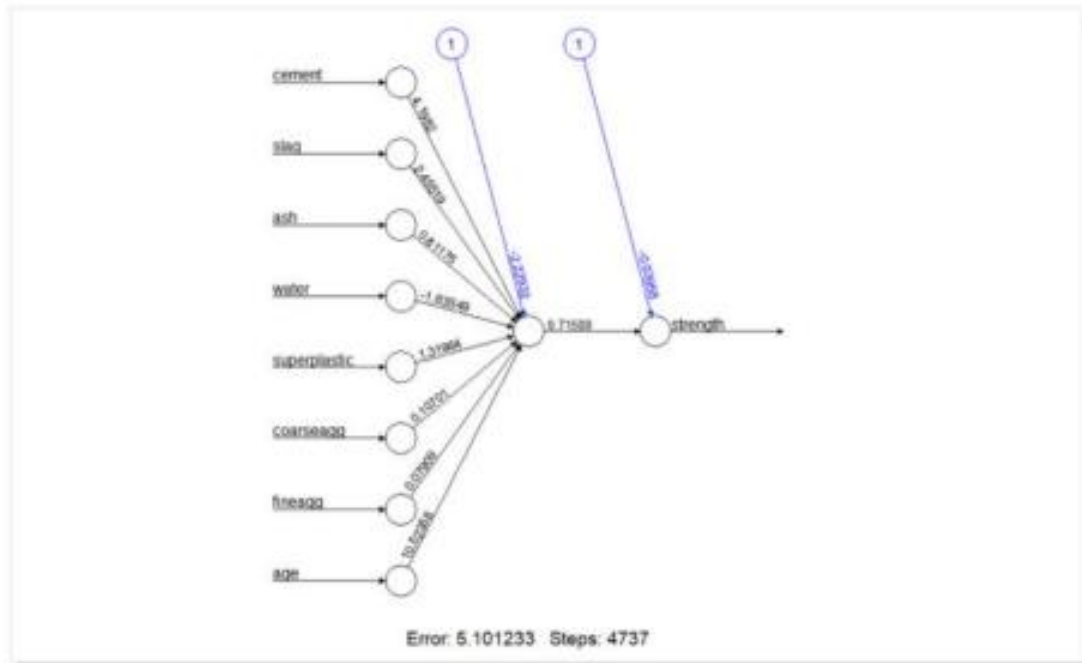
4) 3단계 : 분석 모델 훈련

- 콘크리트 재료와 완성된 산출물의 강도 간의 관계를 모델링함
- 다층 피드포워드 신경망 사용
- neuralnet 패키지 사용
 - * 스테판 프리트쉬(Stefan Fritsch)와 프라우케 겐터(Frauke Guenther)가 구현

인공신경망을 활용한 콘크리트 강도의 모델링

4) 3단계 : 분석 모델 훈련

◦ 단순 다중 네트워크 토폴로지



◦ 표준 신경망 구현

- * 은닉 노드의 기본 설정으로 다층 피드포워드 네트워크 훈련
- * 단순한 모델에서의 특징 단위는 입력 노드가 하나씩 적용됨
- * 그 뒤로 한 개의 은닉 노드와 콘크리트 강도를 예측하는 한 개의 출력 노드 적용
- * 각 단위 연결 에는 가중치 표시
- * 동작점
 - 숫자 1 레이블로 구성된 노드로 표현
 - 숫자 상수, 선형 방정식 등의 해당 노드에서 값이 위쪽 또는 아래쪽으로 이동
- * 훈련 단계 횟수와 오차 제공함 : 예측값에서 빼기, 실제값의 곱
- * SSE가 낮아질수록 예측 성공

인공신경망을 활용한 콘크리트 강도의 모델링

2. 콘크리트 강도 모델링 성능

1) 4단계 : 데이터 모델 성능 평가

- 신경망 다이어그램의 모델이 미래에 적합할지 여부에 대해 많은 정보의 제공이 없음
- 테스트 데이터 셋 예측 생성, `compute()` 함수 사용
- 예측된 콘크리트 강도와 실제값의 상관 관계 측정
→ 두 변수 사이의 선형 연관성 강도의 직관 제공
- 예측과 실제값의 상관성으로 모델링하여 콘크리트 강도에 대한 유용한 척도 측정 → `cor()` 함수 적용
- 상관 관계는 0.93 : 강한 상관 관계
- 한 개의 은닉 노드만 적용
- 모델의 성능 면에서 개선 여지가 있음

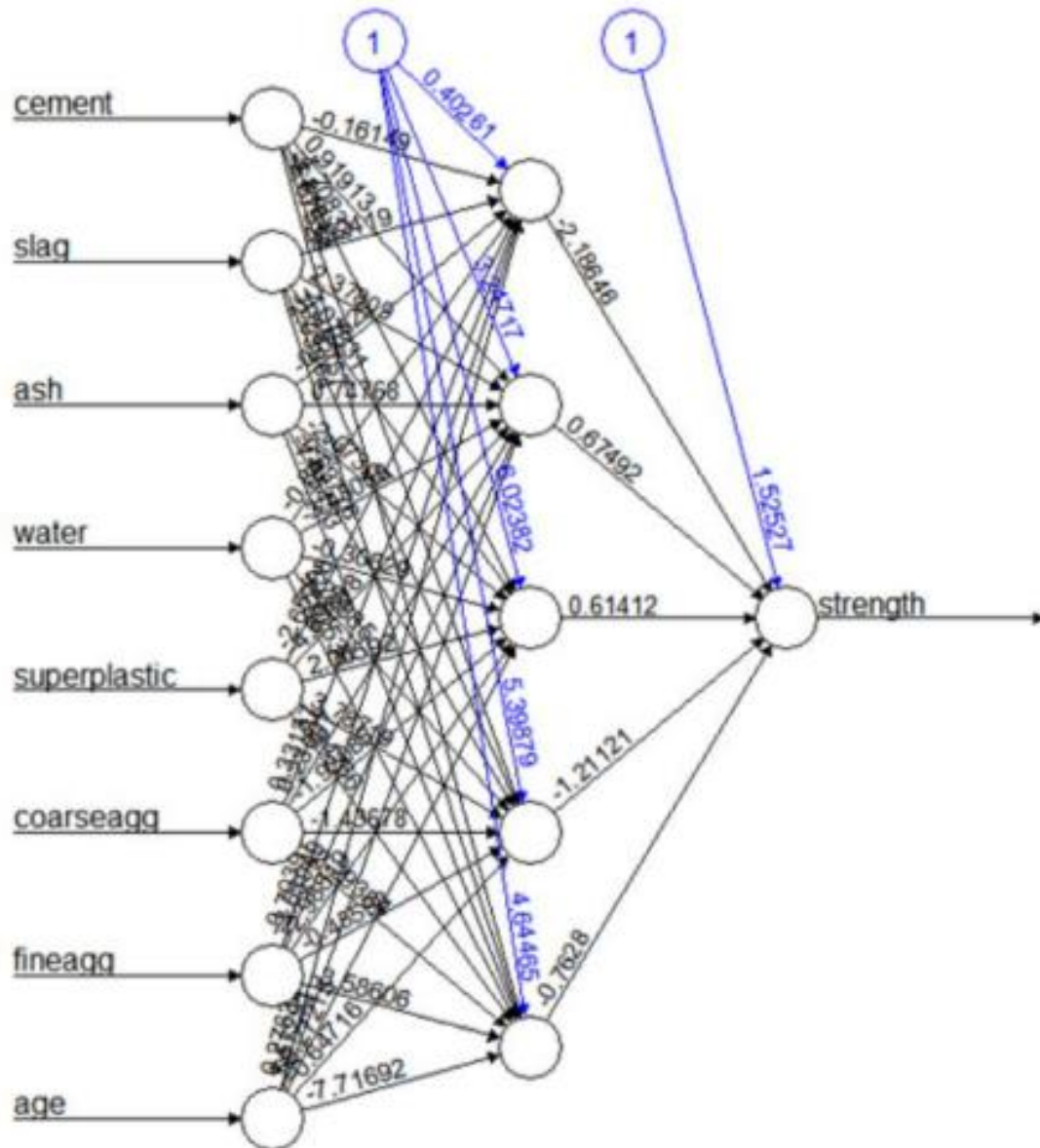
2) 5단계 : 분석 모델 성능 개선

- 추가적인 은닉 계층 도입 또는 망의 활성화 함수 변경
- 변화를 적용했을 경우 심층 신경망의 기초가 생성됨
- 활성화 함수의 선택은 딥러닝에서 중요함
- 예측값과 실제 콘크리트 강도 사이의 상관 관계 계산 = 0.93
→ 예측값과 실제값 : 강한 상관 관계

인공신경망을 활용한 콘크리트 강도의 모델링

2) 5단계 : 분석 모델 성능 개선

- 증가된 은닉 노드 5개 토폴로지의 시각화

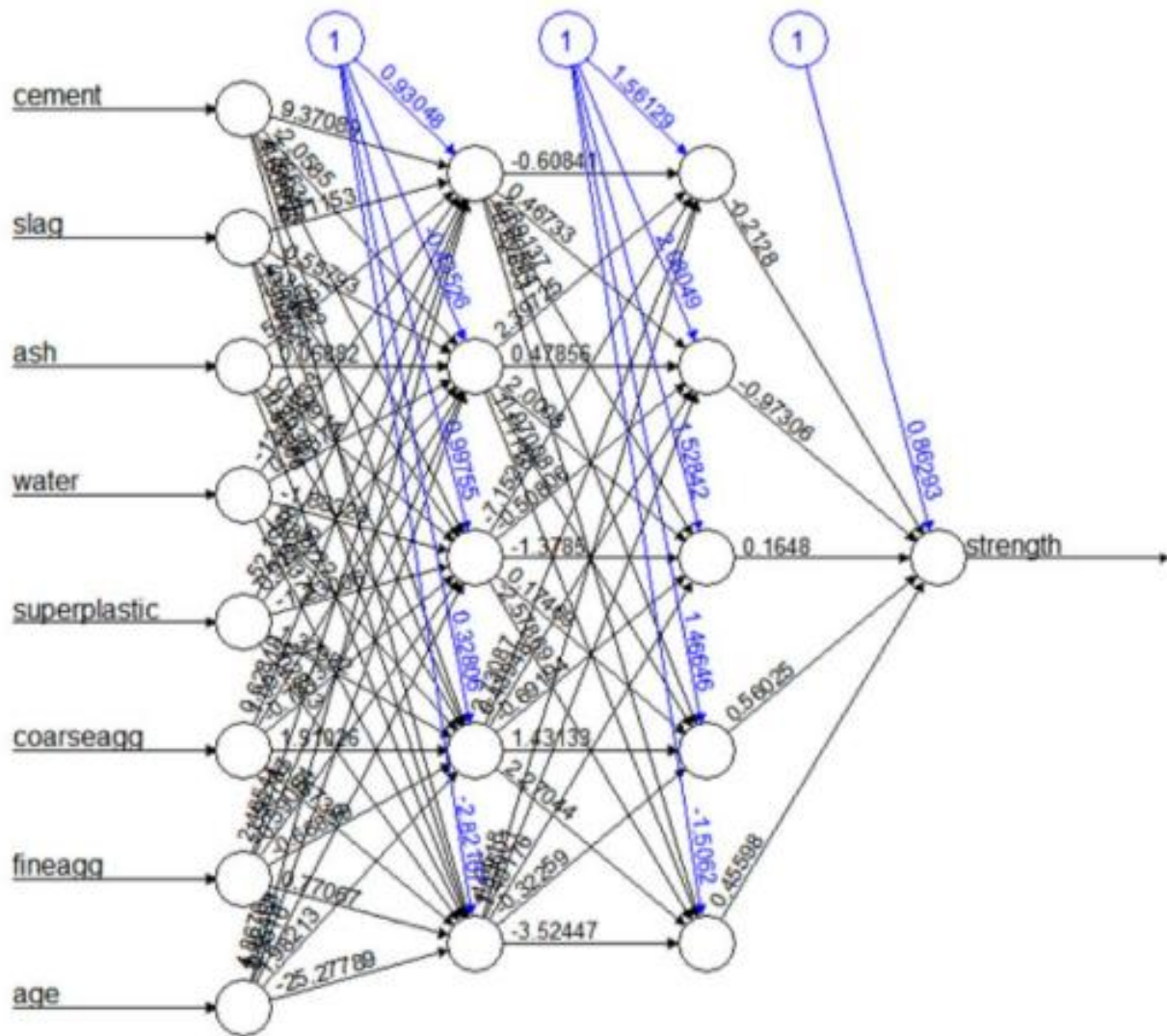


Error: 1.728114 Steps: 36599

인공신경망을 활용한 콘크리트 강도의 모델링

2) 5단계 : 분석 모델 성능 개선

- 활성 함수 은닉 노드 2계층의 네트워크 시각화



Error: 1.511637 Steps: 73411